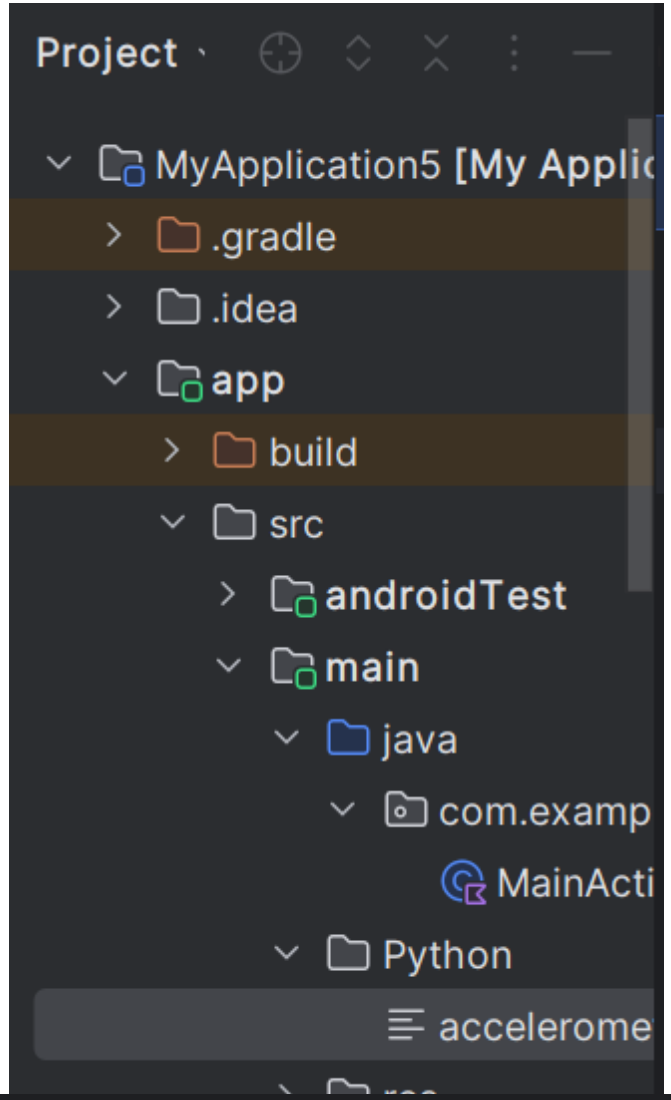
Recording Acceleration Using Mobile Application

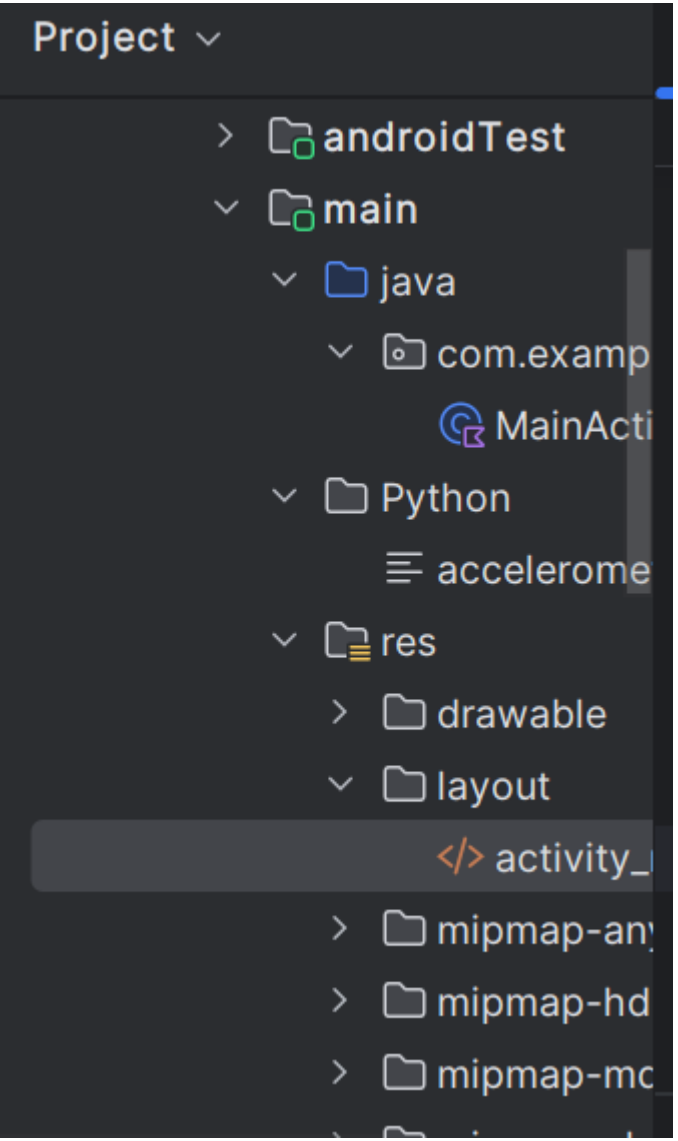
Dr. Salwa Osama

WHAT IS THE ACCELERATION?

Python File



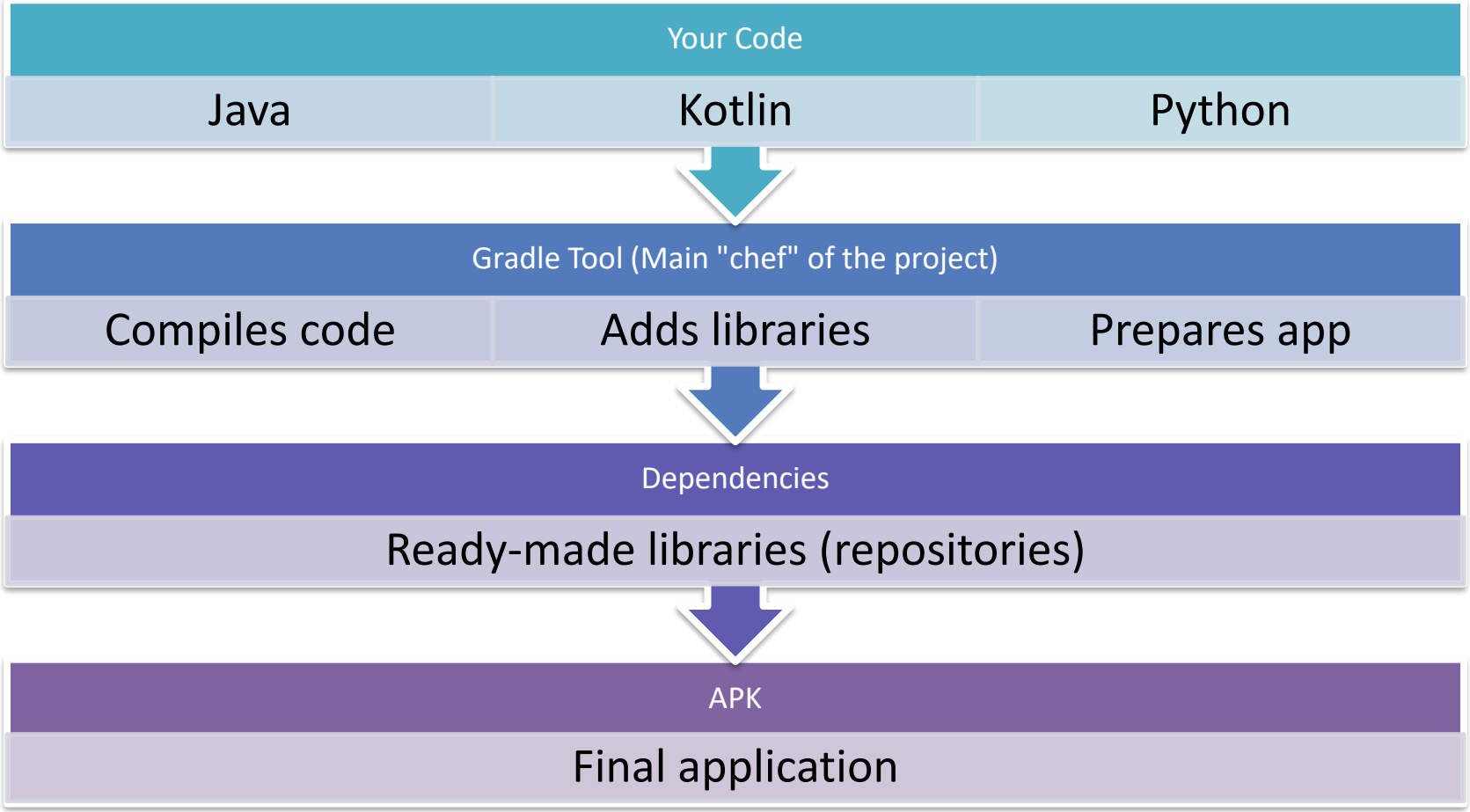

```python
# accelerometer.py
def process_acceleration(x, y, z):
    return f"Acceleration X is: {x:.2f}\nAcceleration Y is: {y:.2f}\nAcceleration Z is: {z:.2f}"
```



Layout .XML

```
<TextView
    android:id="@+id/accelerationText"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello World!"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent" />
```

Gradle


Your Code
Java
Kotlin
Python
Gradle Tool (Main "chef" of the project)
Compiles code
Adds libraries
Prepares app
Dependencies
Ready-made libraries (repositories)
APK
Final application

build.gradle.kts (APP)

This block is responsible for defining the **build settings** themselves, such as the libraries Gradle needs to build the project, and the **repositories** where it will look for these libraries.

```
buildscript {
    repositories {
        google()
        mavenCentral()
        maven( url: "https://chaquo.com/maven")
    }
    dependencies {
        classpath("com.android.tools.build:gradle:8.5.0")
        classpath("com.chaquo.python:gradle:15.0.1")
    }
}
```

build.gradle.kts (APP)

google() → The official Google repository, which contains all the modern Android libraries.
mavenCentral() → A popular repository containing thousands of open-source libraries.
maven("https://chaquo.com/maven") → A repository specifically for the Chaquopy library, which allows us to run Python code inside an Android app.
So any library Gradle needs, it will search for in these repositories.

```
buildscript {
    repositories {
        google()
        mavenCentral()
        maven( url: "https://chaquo.com/maven")
    }
    dependencies {
        classpath("com.android.tools.build:gradle:8.5.0")
        classpath("com.chaquo.python:gradle:15.0.1")
    }
}
```

build.gradle.kts (APP)

```
buildscript {
    repositories {
        google()
        mavenCentral()
        maven( url: "ht
    }
    dependencies {
        classpath("com.android.tools.build:gradle:8.5.0")
        classpath("com.chaquo.python:gradle:15.0.1")
    }
}
```

Here, we define the **libraries that the project needs** for Gradle to run and build the app:

classpath("com.android.tools.build:gradle:8.5.0") → This is the **Android Gradle Plugin**, responsible for converting Android code (Java/Kotlin) into an APK. The version here is 8.5.0.

classpath("com.chaquo.python:gradle:15.0.1") → This is the **plugin for the Chaquopy library**, which allows Android Studio to integrate and run Python code inside the app. The version here is 15.0.1.

build.gradle.kts (Project)

```
> plugins {...}

> andro {...}

> depende {...}
```

```
plugins {
    id("com.android.application")
    id("org.jetbrains.kotlin.android")
    id("com.chaquo.python")
}
```

Defines the **plugins** the project needs:
com.android.application → Marks the project as an Android app.
org.jetbrains.kotlin.android → Enables Kotlin code.
com.chaquo.python → Enables running Python code (Chaquopy).

build.gradle.kts (Project)

```
> plugins {...}

> android {...}

> dependencies {...}
```

```
android {
    namespace = "com.example.myapplication"
    compileSdk = 34

    defaultConfig {
        applicationId = "com.example.myapplication"
        minSdk = 26
        targetSdk = 34
        versionCode = 1
        versionName = "1.0"

        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"

        ndk {
            abiFilters += listOf("armeabi-v7a", "arm64-v8a", "x86", "x86_64")
        }
```

Main Android configuration block:
namespace → Project package name.
compileSdk → Android version to build against.
defaultConfig { ... } → Basic app settings:
- applicationId → Unique app identifier.
- minSdk → Minimum Android version supported.
- targetSdk → Target Android version for compatibility.

build.gradle.kts (Project)

```
> plugins {...}

> android {...}

> dependencies {...}
```

```
dependencies {
    implementation("androidx.core:core-ktx:1.8.0")
    implementation("androidx.appcompat:appcompat:1.5.1")
    implementation("com.google.android.material:material:1.7.0")
    implementation("androidx.activity:activity:1.6.1")
    implementation("androidx.constraintlayout:constraintlayout:2.1.4")

    testImplementation("junit:junit:4.13.2")
    androidTestImplementation("androidx.test.ext:junit:1.1.4")
    androidTestImplementation("androidx.test.espresso:espresso-core:3.5.0")

    // Kotlin Standard Library
    implementation("org.jetbrains.kotlin:kotlin-stdlib:1.9.10")
}
```

Lists all libraries the app depends on:
Core Android libraries (core-ktx, appcompat, material, constraintlayout)
Testing libraries (junit, espresso, androidx.test)
Kotlin Standard Library (kotlin-stdlib)

MainActivity.kts

```
import android.hardware.Sensor
import android.hardware.SensorEvent
import android.hardware.SensorEventListener
import android.hardware.SensorManager
import android.os.Bundle
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity
import com.chaquo.python.Python
import com.chaquo.python.android.AndroidPlatform
```

Sensor, SensorEvent, SensorEventListener, SensorManager → To work with device sensors.
TextView → To display values on the screen.
Python, AndroidPlatform → To run Python code inside the app.

MainActivity.kts

```
class MainActivity : AppCompatActivity(), SensorEventListener {

    private lateinit var sensorManager: SensorManager
    private var accelerometer: Sensor? = null
    private lateinit var accelerationText: TextView
```

Inherits from AppCompatActivity and implements SensorEventListener to receive sensor data.
sensorManager → Manages the device's sensors.
accelerometer → The accelerometer sensor.
accelerationText → TextView to show results.

MainActivity.kts

```
class MainActivity : AppCompatActivity(), SensorEventListener {

    private lateinit var sensorManager: SensorManager
    private var accelerometer: Sensor? = null
    private lateinit var accelerationText: TextView
```

Inherits from AppCompatActivity and implements SensorEventListener to receive sensor data.
sensorManager → Manages the device's sensors.
accelerometer → The accelerometer sensor.
accelerationText → TextView to show results.

MainActivity.kts

```
class MainActivity : AppCompatActivity(), SensorEventListener {

    private lateinit var sensorManager: SensorManager
    private var accelerometer: Sensor? = null
    private lateinit var accelerometerText: TextView
```

sensorManager
It is an object from the type SensorManager
It **controls and manages all sensors** in the device (like accelerometer, gyroscope…)
lateinit means:
We will give this variable a value later (inside onCreate()), not now.

MainActivity.kts

```
class MainActivity : AppCompatActivity(), SensorEventListener {

    private lateinit var sensorManager: SensorManager
    private var accelerometer: Sensor? = null
    private lateinit var accelerationText: TextView
```

accelerometer: Sensor? = null
This is the actual **accelerometer sensor**
The ? means the value may be **null**
(because not all phones have this sensor)
We start with null and then check if the sensor exists in the device

MainActivity.kts

```
class MainActivity : AppCompatActivity(), SensorEventListener {

    private lateinit var sensorManager: SensorManager
    private var accelerometer: Sensor? = null
    private lateinit var accelerationText: TextView
```

accelerationText: TextView
This is the text area on the screen that will show the acceleration values
Also uses lateinit because:
We will connect it to the layout (XML) after setContentView()

MainActivity.kts

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)
    // Initialize Python
    if (!Py  on.isStarted()) {
        Pyth  tart(AndroidPlatform( context: this))
    }
    // Initial  tView
    acceleratio  findViewById(R.id.accelerationText)
    // Initializ  anager
    sensorManager  mService(SENSOR_SERVICE) as SensorManager
    accelerometer  ger.getDefaultSensor(Sensor.TYPE_ACCELEROMETER)
    accelerometer?.
        sensorManage  ner( listener: this, it, SensorManager.SENSOR_DELAY_NORMAL)
```

This tells Android which XML layout file to show on the screen.

MainActivity.kts

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)
    // Initialize Python
    if (!Python.isStarted()) {
        Python.start(AndroidPlatform( context: this))
    }
    // Initialize TextView
    accelerationText = findViewById(R.id.accelerationText)
    // Initialize Sensor Manager
    sensorManager = getS  mService(SENSOR_SERVICE) as SensorManager
    accelerometer = sen  ger.getDefaultSensor(Sensor.TYPE_ACCELEROMETER)
    accelerometer?.let
        sensorManager.r  ner( listener: this, it, SensorManager.SENSOR_DELAY_NORMAL)
```

This line finds the TextView in your layout so you can update it with the sensor values.

MainActivity.kts

```kotlin
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)
    // Initialize Python
    if (!Python.isStarted()) {
        Python.start(AndroidPlatform( context: this))
    }
    // Initialize TextView
    accelerationText = findViewById(R.id.accelerationText)
    // Initialize SensorManager
    sensorManager = getSystemService(SENSOR_SERVICE) as SensorManager
    accelerometer = sensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER)
    accelerometer?.let {
        sensorManager.registerListener( listener: this, it, SensorManager.SENSOR_DELAY_NORMAL)
```

This gets the phone’s sensor system to work with sensors like Accelerometer.

MainActivity.kts

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)
    // Initialize Python
    if (!Python.isStarted()) {
        Python.start(AndroidPlatform( context: this))
    }
    // Initialize TextView
    accelerationText = findViewById(R.id.accelerationText)
    // Initialize SensorManager
    sensorManager = getSystemService(SENSOR_SERVICE) as SensorManager
    accelerometer = sensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER)
    accelerometer?.let {
        sensorManager.regi[illegible]ner( listener: this, it, SensorManager.SENSOR_DELAY_NORMAL)
```

This asks the system for the Accelerometer sensor (which measures phone movement).

MainActivity.kts

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)
    // Initialize Python
    if (!Python.isStarted()) {
        Python.start(AndroidPlatform( context: this))
    }
    // Initialize TextView
    accelerationText = findViewById(R.id.accelerationText)
    // Initialize SensorManager
    sensorManager = getSystemService(SENSOR_SERVICE) as SensorManager
    accelerometer = sensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER)
    accelerometer?.let {
        sensorManager.registerListener( listener: this, it, SensorManager.SENSOR_DELAY_NORMAL)
```

If the accelerometer exists (not null)
Start listening for movement values
Update the values regularly (SENSOR_DELAY_NORMAL)
So now the app will get updates whenever the phone moves — you will show these values in the TextView.

MainActivity.kts

```
override fun onSensorChanged(event: SensorEvent?) {
    event?.let {
        val x = it.values[0
        val y = it.values[1]
        val z = it.values[2]
        // Send values to Pyth
        val py = Python.getInst
        val module = py.getModul        "accelerometer")  //  Python: accelerometer.py
        val result = module.callA        process_acceleration", x, y, z).toString()
        // Show result on screen
        accelerationText.text = res
    }
```

This function is called **automatically** every time the phone moves.
The accelerometer sends new values for movement on **X, Y, and Z axes**.

MainActivity.kts

```
override fun onSensorChanged(event: SensorEvent?) {
    event?.let {
        val x = it.values[0]
        val y = it.values[1]
        val z = it.values[2]
        // Send values to Python
        val py = Python.getInstance()
        val module = py.getModule( name: "accelerometer")  //  Python: accelerometer.py
        val result = module.callAttr( key: "process_acceleration", x, y, z).toString()
        // Show result on screen
        accelerationText.text = result
    }
```

Check if sensor data exists

MainActivity.kts

```
override fun onSensorChanged(event: SensorEvent?) {
    event?.let {
        val x = it.values[0]
        val y = it.values[1]
        val z = it.values[2]
        // Send values to Pyt n
        val py = Python.getIns nce()
        val module = py.getMod ( name: "accelerometer")  //  Python: accelerometer.py
        val result = module.ca tr( key: "process_acceleration", x, y, z).toString()
        // Show result on scree
        accelerationText.text =  t
    }
```

Read movement values from the sensor

MainActivity.kts

```kotlin
override fun onSensorChanged(event: SensorEvent?) {
    event?.let {
        val x = it.values[0]
        val y = it.values[1]
        val z = it.values[2]
        // Send values to Python
        val py = Python.getInstance()
        val module = py.getModule( name: "accelerometer")  //  Python: accelerometer.py
        val result = module.callAttr( key: "process_acceleration", x, y, z).toString()
        // Show result on screen
        accelerationText.text = result
    }
```

py → get Python engine already runninggetModule("accelerometer") → open file accelerometer.py in your Python folderprocess_acceleration → call a Python function and pass x, y, z valuesresult → whatever Python returns (as text)

MainActivity.kts

```
override fun onSensorChanged(event: SensorEvent?) {
    event?.let {
        val x = it.values[0]
        val y = it.values[1]
        val z = it.values[2]
        // Send values to Python
        val py = Python.getInstance()
        val module = py.getModule( name: "accelerometer")  //  Python: accelerometer.py
        val result = module.callAttr( key: "process_acceleration", x, y, z).toString()
        // Show result on screen
        accelerationText.text = result
    }
```

This updates the TextView on your screen to show the value that Python calculated.

Thanks